Day 12 — Git & GitHub, from the beginning

1. First, understand the basic concepts

Git

Git tracks changes to your code on your computer. It lets you see what changed and return to earlier versions.

GitHub

GitHub is an online platform where you can store your code, share it with others, and work together.

Branch

A separate line of work where you can make changes without directly changing the `main` branch.

Pull Request (PR)

A request to review your changes and merge them into another branch, usually `main`.

Imagine you are writing a college assignment

- `main` = your original, stable assignment.
- `feature/hello-message` = a separate copy where you add something new.
- `commit` = saving a named checkpoint of your work.
- `push` = uploading your branch to GitHub.
- `Pull Request` = asking to include your changes in the main assignment.
- `merge` = combining your changes into `main`.
- `pull` = downloading the latest changes from GitHub to your computer.

2. Understand the complete workflow

1. Edit your code

Change `hello.py` in VS Code

2. Create a branch

Work separately from `main`

3. Stage and commit

Choose the changes and save a checkpoint

4. Push to GitHub

Upload your branch online

5. Create a Pull Request

Review and merge the changes

6. Update local `main`

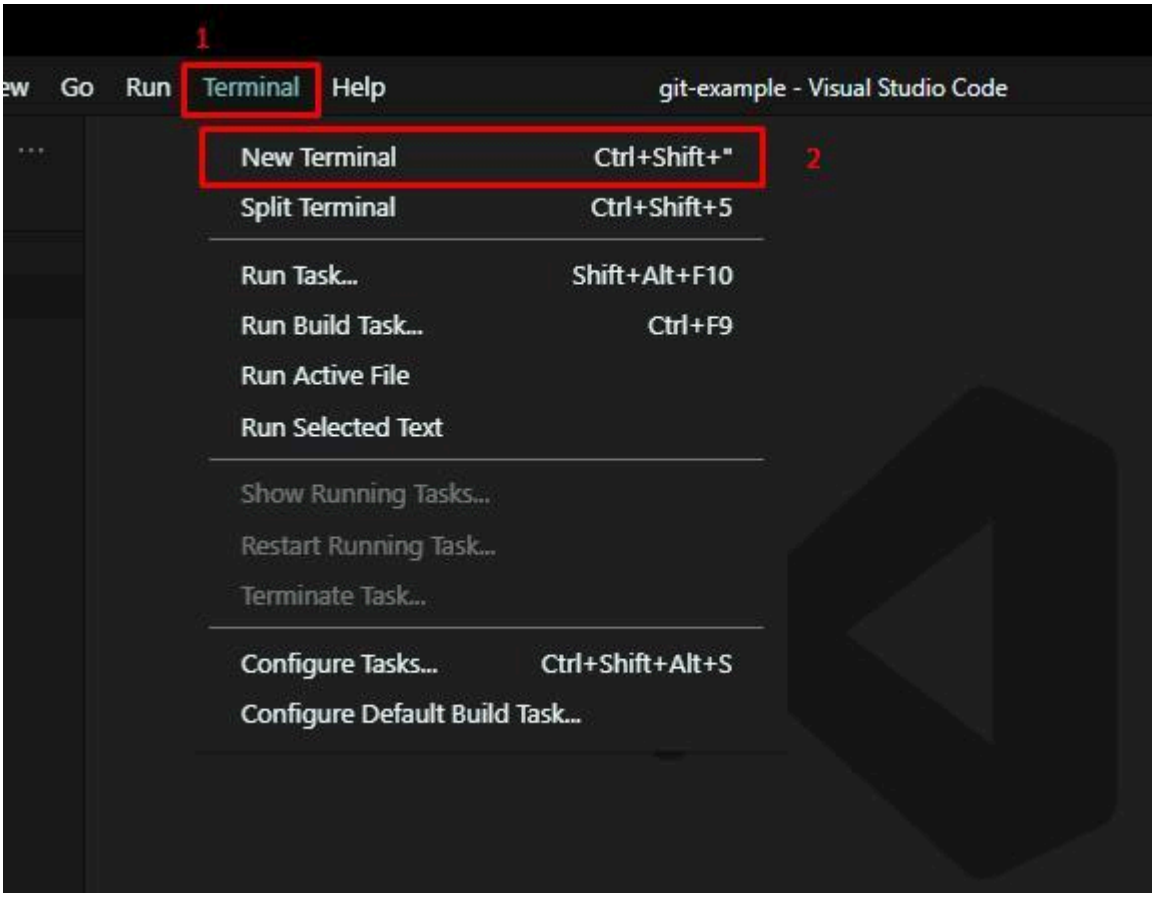
Download the merged changes and verify

This is the process we'll follow. You don't have to memorise it immediately.

3. Step-by-step practical: Let's do it in VS Code

We'll use your existing Git project if you already created it. Don't create another folder yet.

Step 1 — Open the VS Code terminal



Where to click

1. Open VS Code.
2. Open your existing Git practice project folder.
3. From the top menu, click Terminal → New Terminal.
4. A terminal opens at the bottom.

The terminal is where you type Git commands. You do not type these commands inside your Python file.

For example, your terminal might look like this:

PS C:\Users\YourName\your-project>

The path will be different on your computer, and that's fine.

Step 2 — Check your current Git status

Type this command in the terminal and press Enter:

git status

What does it do?

It tells you:

- Which branch you're currently using.
- Whether you have unsaved changes from Git's perspective.
- Whether any files need to be committed.

You might see:

On branch main
nothing to commit, working tree clean

This means your current branch is `main`, and Git has no uncommitted changes.

Or you might see:

On branch feature/hello-message

That means you're already on your feature branch.

Important: Don't worry if your output is different. We should follow what your terminal actually says.

Step 3 — Understand branches

Run:

git branch

Example output:

* main

The `*` tells you which branch you're currently using.

If you see:

 main
* feature/hello-message

You're on `feature/hello-message`.

Now, what if we want to create a new branch?

git switch -c feature/hello-message

Let's break this command down:

Part	Meaning
`git`	Use Git
`switch`	Change the current branch
`-c`	Create a new branch
`feature/hello-message`	The new branch's name

Run this only if you're on `main`, your working tree is clean, and the branch doesn't already exist.

If the branch already exists, use:

git switch feature/hello-message

If you already completed this step, you don't need to repeat it.

Why create a branch? Because you can work on a new feature without directly modifying `main`.

Step 4 — Edit your Python file

In VS Code, open the existing `hello.py` file.

Keep the following four lines in the file:

print("Hello Git")print("Learning version control")print("I am learning Git step by step")print("Practicing branches and pull requests")

Save the file using Ctrl + S.

What does this code do?

- `print()` displays text on the screen.
- Each line prints one message.
- Python executes the statements from top to bottom.

Your expected output is:

Hello Git
Learning version control
I am learning Git step by step
Practicing branches and pull requests

Run the file

In your VS Code terminal, type:

python hello.py

If your system doesn't recognise `python`, try:

py hello.py

This runs your Python file. It does not upload anything to GitHub or save a Git commit.

Step 5 — See what you changed

Now run these commands one at a time:

git status

This checks your repository's current state.

Next:

git diff

This shows changes in tracked files compared with the last committed version.

For example, if you added a line, the output might contain:

+ print("Practicing branches and pull requests")

The `+` indicates an added line. A `-` indicates a removed line.

If `git diff` displays nothing, your file might already match the last commit, or the file may not be tracked yet.

Step 6 — Stage your changes

Type:

git add hello.py

What does this mean?

Imagine you've edited five files but only want to include one in your next checkpoint.

`git add hello.py` tells Git to include the changes to `hello.py` in the next commit.

Now check:

git status

You might see:

Changes to be committed:
  modified: hello.py

This means the change is staged and ready to commit.

Remember: `git add` does not upload your file to GitHub.

Step 7 — Commit your changes

Run:

git commit -m "Add feature branch message"

Breaking it down:

- `git commit` creates a saved checkpoint.
- `-m` lets you add a message describing that checkpoint.
- `"Add feature branch message"` explains what you changed.

You might see:

[feature/hello-message abc1234] Add feature branch message
1 file changed

The exact output will vary.

Your change is now committed in your local Git repository.

If Git asks you to configure your name and email, or says there is nothing to commit, stop and check the message before proceeding.

Step 8 — Push your branch to GitHub

Now type:

git push -u origin feature/hello-message

Let's understand every part:

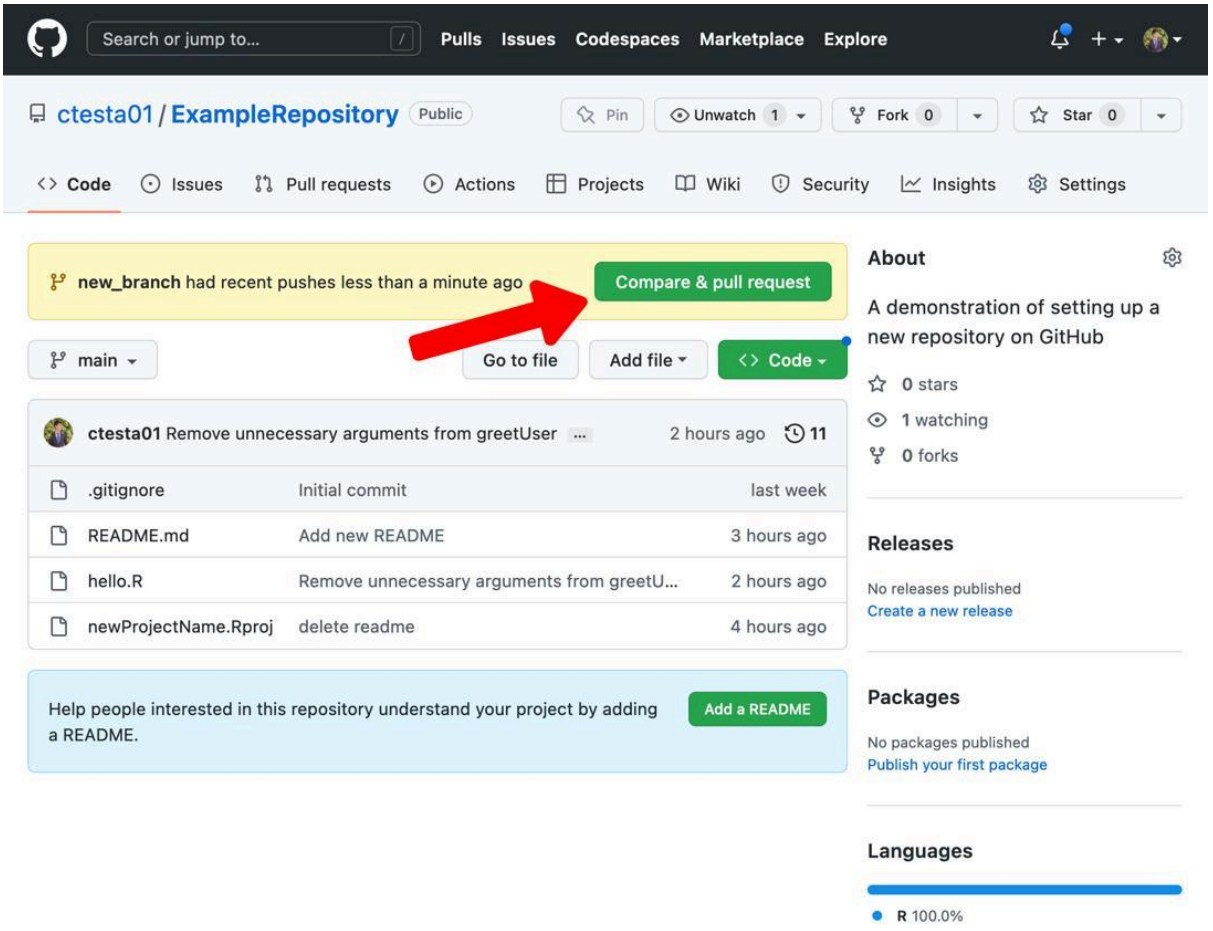
Part	Meaning
`push`	Upload your committed changes
`-u`	Set the upstream branch for future pushes
`origin`	The usual name for your remote GitHub repository
`feature/hello-message`	The branch you want to upload

For this command to work, your project must already be connected to a GitHub repository called `origin`, and the branch must exist locally.

After a successful push, your feature branch will be available on GitHub.

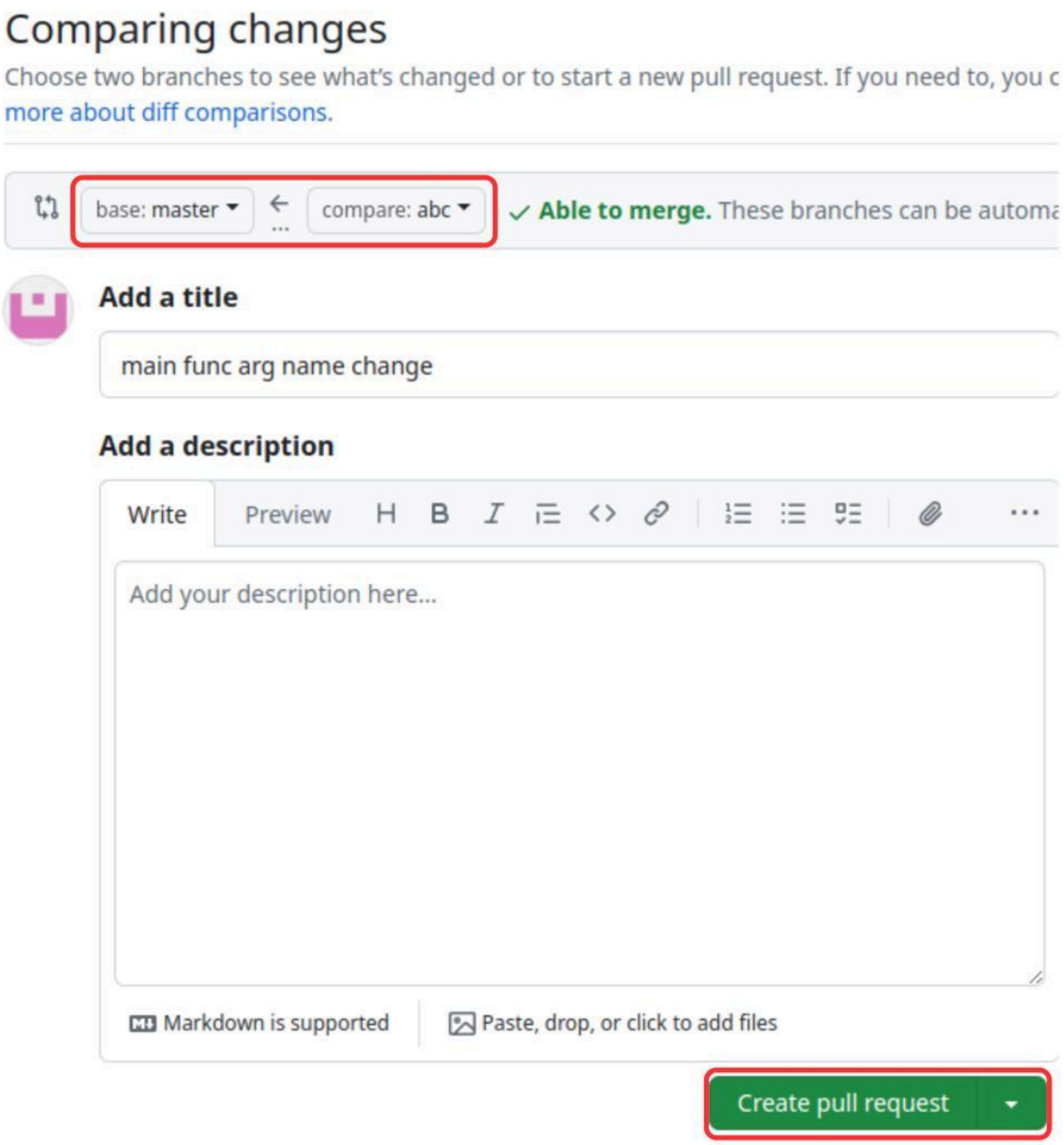
If you see an error about `origin`, authentication, or a rejected push, don't use force-push. Share the exact error so we can fix it safely.

Step 9 — Create a Pull Request on GitHub



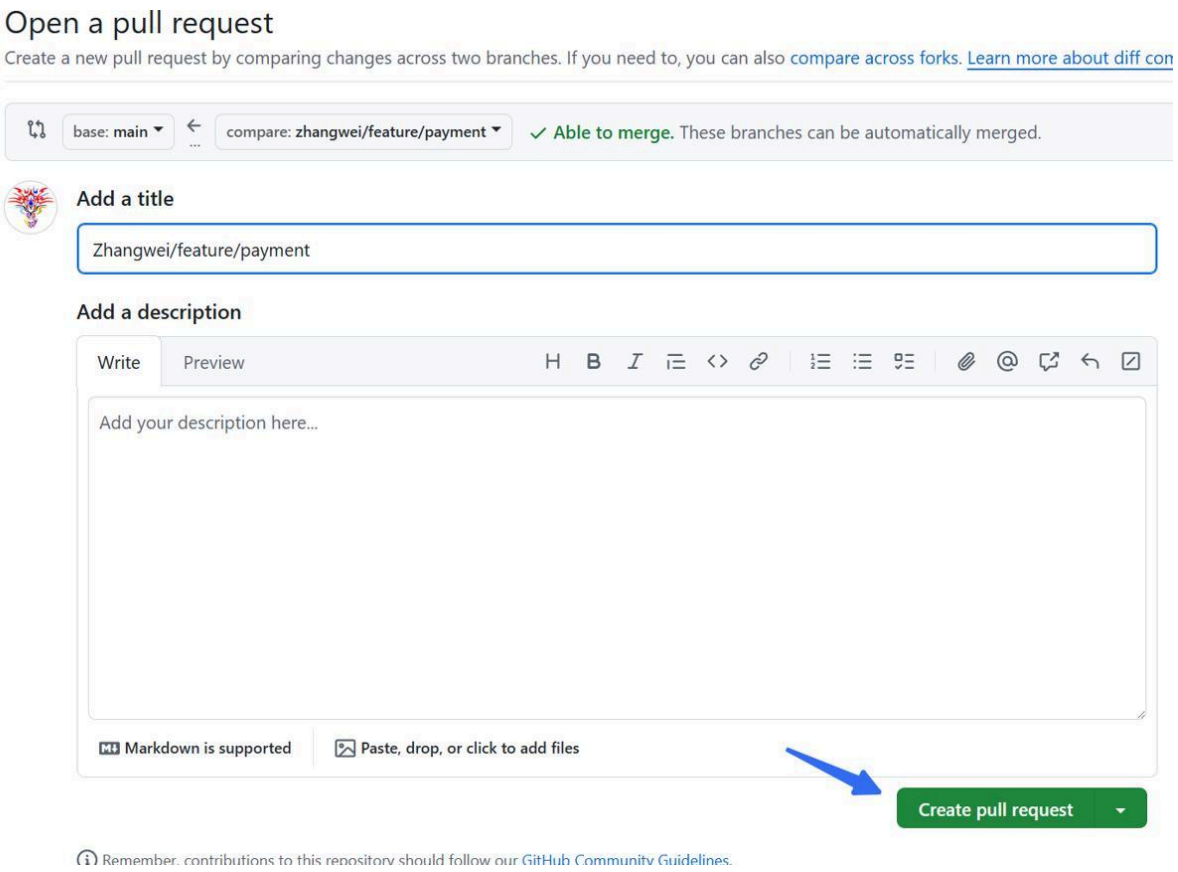
9.1 Open your repository

Go to GitHub in your browser and open the repository connected to your project. Find your uploaded feature branch.



9.2 Start a Pull Request

Click Compare & pull request, or open the Pull requests tab and choose New pull request.



9.3 Select the branches

Set the branches as follows:

- Base: `main`
- Compare: `feature/hello-message`

Review the displayed changes, then click Create pull request.

Use this title:

Add feature branch message

A Pull Request lets you review your proposed changes before they become part of `main`.

What happens when you merge?

When the changes are approved and you click Merge pull request, GitHub incorporates the feature branch's changes into `main`.

For your practice project, you can merge your own PR if GitHub allows it and there are no required reviews or checks. In a real team, you would follow the team's review rules.

Step 10 — Update your local `main` branch

After the Pull Request has been merged on GitHub, return to the VS Code terminal.

First, switch to `main`:

git switch main

Next, download and integrate the latest changes from GitHub:

git pull origin main

What is the difference between these two commands?

- `git switch main` changes your current local branch.
- `git pull origin main` fetches updates from the remote `main` branch and integrates them into your local `main`.

If Git reports uncommitted changes, don't force anything. Resolve or preserve those changes before switching branches.

Step 11 — Verify everything

Run these commands one by one:

python hello.py

This checks that the Python file runs.

git log --oneline --graph --all

This shows your commit history in a compact graph.

git status

Ideally, you should see:

On branch main
nothing to commit, working tree clean

Your file should print the four expected messages.

4. The most important Git commands to remember

Command	Simple meaning

`git status`	Check what's happening
`git branch`	List local branches
`git switch main`	Switch to `main`
`git switch -c branch-name`	Create and switch to a branch
`git diff`	View uncommitted changes to tracked files
`git add hello.py`	Stage the file's changes
`git commit -m "message"`	Save a checkpoint
`git push`	Upload commits
`git pull`	Get and integrate remote changes
`git log --oneline`	View commit history

One more important concept: Git normally tracks the saved contents of files, so remember to save your file in VS Code before reviewing or staging changes.